

NETWORK ARCHITECTURE AND PROTOCOLS (NAP)

WORKSHOP 2

Open files and data interchange
formats

W2. Open files and data interchange formats		Estimated Time:	4 hours
		Date:	Sep 2026
Students' Name:	Mathis TRUMEAU		

CONTENTS

CONTENTS.....	2
OVERVIEW.....	3
LINUX.....	4
OPEN STANDARDS FOR DATA.....	4
JSON.....	5
YAML.....	8
CSV.....	10
MONGODB.....	10
INSTALL MONGODB COMMUNITY EDITION.....	11
MONGO SHELL.....	13
MONGO COMPASS.....	13
WORKSHOP ACTIVITIES.....	15
1) Create a JSON file from scratch (30%).....	15
2) Create a YAML file from JSON (20%).....	16
3) Create a MongoDB document from JSON (30%).....	18
4) Create a JSON file from MongoDB (20%).....	19
FINAL COMMENTS.....	21

OVERVIEW

Objectives:

- Get familiar with open standards for data exchange.
- Create a json file as a common case of data interchange in 5G.
- Create a yaml file as a common case of configuration file in 5G.
- Learn how MongoDB organizes and stores data.
- Produce a MongoDB document from a json file.
- Create a csv file from a MongoDB collection.
- Get familiar with open-source tools to manage open data.

Sources:

- **Open Standards for Data**, Guide created by the Open Data Institute:
<https://standards.theodi.org>
- **JSON**, Official JSON webpage:
<https://www.json.org/>
- **YAML**, Official YAML webpage:
<https://yaml.org/>
- **CSV**, Common Format and MIME Type for CSV Files:
<https://www.ietf.org/rfc/rfc4180.txt>

Equipment:

- **Personal Computer:**
 - o Intel Core, i5, min 16 GB RAM.
 - o Linux OS, UBUNTU v20 or higher / DEBIAN v10 or higher.
 - o MongoDB, ver 5 or higher.
 - o MongoDB Compass, ver 1.28 or higher.
 - o yq, ver 4.40 or higher.

LINUX

Computers in Master's laboratory come with two operative systems (OS) already installed: Ubuntu/Debian and Windows. The **GRand Unified Bootloader** (GRUB) is initially started for you to select the operative system to be loaded.

As a generic user password, use: 123456, to enter both the work desktop. By default, your username should be "LabSistemas".

First step is to login as root user and "update + upgrade" the installed packages:

```
$ sudo su  
$ apt update  
$ apt upgrade
```

OPEN STANDARDS FOR DATA

Standards are documented agreements that solve a specific set of problems or meet clearly defined needs. They are used in industries and sectors across the world to document agreements on physical items, ideas, digital products, processes, and more.

In words of the ITU, **open standards** are standards made available to the public and are developed (or approved) and maintained via a collaborative and consensus-driven process. This definition implies that open standards are not just available for anyone to read and implement, but the process of creating them is itself open to participation.

Consequently, **open standards for data** are reusable agreements that make it easier for people and organisations to publish, access, share and use better quality data. As a result, outputs of an open standard for data can be data themselves or a model.

There are thousands of open standards for data in use around the world but, generally, they can be classified in according to the next three broad purposes:

- **Share vocabularies** and common language using common models, attributes and definitions, with outputs as: registers, taxonomies, vocabularies and ontologies.
- **Exchange data** within and between organisations and systems using common formats and shared rules, with outputs as: specifications, schemas and templates.
- **Provide guidance and recommendations** for sharing better quality data, understanding processes and information flow, with outputs as: models, protocols, and guides

All open standards share common features, for example being available for anyone to access, use or share. Depending on the purpose and products of a given open standard, some features are more relevant than others. For example, it is important when using a data exchange standard to check that data has been produced correctly by checking data against the standard's rules.

Open data is only useful if it is shared in ways that people can easily understand. It needs to be shared in a standardised format, with clear rules, and easily traced back to where it came from.

Particularly, standards for **exchanging data** (the second group abovementioned) specify common formats and shared rules that lead to consistent data. A good standard for data exchange avoids problems, skips misunderstandings, and provides tools to check that data has been properly structured.

Typical **data exchange standards** define a common format for data that describes how they should be serialised or structured for sharing. Or it might combine common formats, shared vocabularies and other rules to describe what data should be shared to solve a specific problem.

JSON

JSON (JavaScript Object Notation), pronounced /'dʒeɪsən/, is an open standard file and data interchange format that uses human-readable text to store and transmit data objects consisting of **attribute-value** pairs and **arrays**. It is a common data format with diverse uses in electronic data interchange. For example, in the 5G core network, JSON is the file format employed to interchange information between network functions in a **service-based architecture** (SBA). An example of a data packet interchanged in a 5G mobile network is shown next.

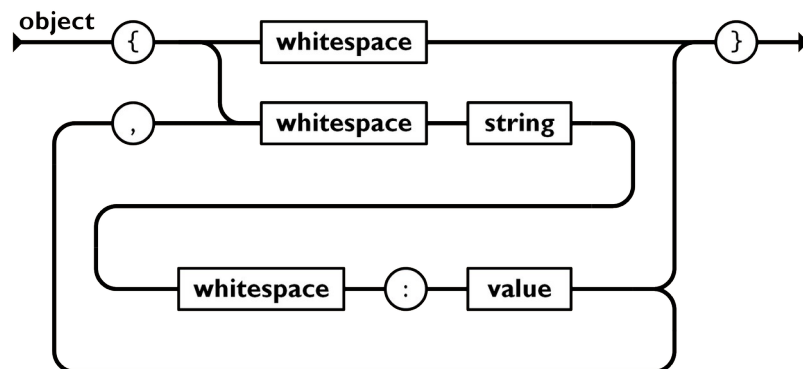
DATA:

```
{
  "authType" : "5G_AKA", "authenticationVector": {"avType": "5G_HE_AKA",
    "rand" : "87d3bc51f51f058c36057d088ce9fa72",
    "xresStar" : "35e6b894eac81bcd3a6753a1ba8d3be9",
    "autn" : "62b14a63fb9f900187c29e62b16bf3fa",
    "kausf" : "fe6ad77e9341e377888a31682f86ac995d20b340049eba3a8e1082e11b95ed6b" }
}
```

JSON was derived from JavaScript, but many modern programming languages include code to generate and parse JSON-format data. JSON is built on two formal structures:

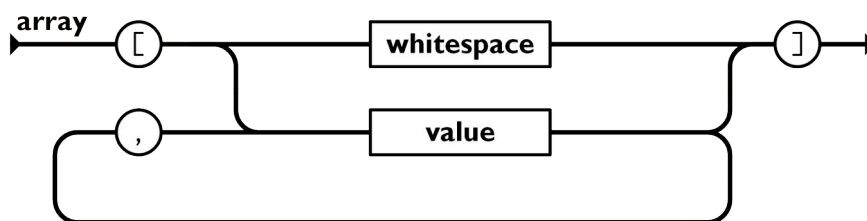
- A collection of **name/value pairs**. In various languages, this is realized as an **object**, record, struct, dictionary, hash table, keyed list, or associative array.
- An ordered **list of values**. In most languages, this is realized as an **array**, vector, list, or sequence.

An **object** is an unordered set of name/value pairs. An object begins with a left brace ({) and ends with a right brace (}). Each name is followed by a colon (:) and its value, and the name/value pairs are separated by a comma (,).



For example: `{"firstname" : "Tom" , "surname" : "Cruise"}`.

An **array** is an ordered collection of objects. An array begins with left bracket ([) and ends with right bracket (]). Values are separated by comma (,).



For example: `{ [ {object} , {object} ] }`.

At both, objects and arrays, the **value** can be a string in double quotes, or a number, or a logical true or false, or null, or an object, or even an array. These structures can be nested.

Likewise, a **string** is a sequence of zero or more Unicode characters, wrapped in double quotes, using backslash escapes. A character is represented as a single character string. A string is very much like a C or Java string. A **number** is very much like a C or Java number, except that the octal and hexadecimal formats are not used. Let's see some examples:

1) JSON string example:

```
1 {
2   "firstname": "Tom",
3   "lastname": "Cruise",
4   "occupation": "Actor"
5 }
```

2) JSON number example:

```
1 {
2   "id": 1,
```



```
3  "age": 56,  
4  "bornyear": 1963,  
5  "date": 3,  
6  "month": 7,  
7  "weight" : 67.5  
8 }  
9
```

3) JSON object example:

```
1 {  
2   "actor": {  
3     "name": "Tom Cruise",  
4     "age": 56,  
5     "birthplace": "Syracuse, NY",  
6     "birthdate": "July 3 1962",  
7     "photo": "https://jsonformatter.org/img/tom-cruise.jpg"  
8   }  
9 }  
10
```

4) JSON array (list) example:

```
1 {  
2   "actors": [  
3     {  
4       "name": "Tom Cruise",  
5       "age": 56,  
6       "birthplace": "Syracuse, NY",  
7       "birthdate": "July 3, 1962",  
8       "photo": "https://jsonformatter.org/img/tom-cruise.jpg"  
9     },  
10    {  
11      "name": "Robert Downey Jr.",  
12      "age": 53,  
13      "birthplace": "New York City, NY",  
14      "birthdate": "April 4, 1965",  
15      "photo": "https://jsonformatter.org/img/Robert-Downey-Jr.jpg"  
16    }  
17  ]  
18 }  
19
```

YAML

YAML (Yet Another Markup Language), pronounced /'jæm.ɪ/, is a digestible data serialization language often used to create configuration files with any programming language. That is the case for OPEN5GS and UERANSIM, where configuration of network functions in the core network and configuration of gNB and UE, is done through their respective YAML files.

Designed for human interaction, YAML is a strict superset of JSON. That is the reason why it can do everything that JSON can do and more. A major difference is that newlines and indentation actually mean something in YAML, as opposed to JSON, which uses brackets and braces.

The basic structure of a YAML file is a **map**. You might call this a dictionary, hash or object, depending on your programming language or mood.

The most typical definition in a YAML file is a **key-value pair**, that is a key that points to a string value. For example: `mcc: '999'`. It is also admitted integers, floating-point numbers, Booleans, etc. But YAML also supports more than just string values like arrays or nesting of key-values, and mixing types.

Whitespace is part of YAML's formatting. Unless otherwise indicated, newlines indicate the end of a field. You structure a YAML document with indentation. The indentation level can be one or more spaces (a best use is two spaces). The specification forbids tabs because tools treat them differently.

In YAML user comments are allowed. They begin with a pound sign (#) and can appear after a value in the document or take up an entire line.

YAML uses three main data formats:

- **Scalars**: the simplest in a key:value view.
- **List/sequence**: data ordered by indexes.
- **Dictionary/mapping**: similar to scalars but can contain nested data including other data types.

Let's see some examples:

1) YAML scalars:

```
1 ---
2 firstname: "Tom"
3 lastname: "Cruise"
4 occupation: "Actor"
5 age: 56
```

2) YAML lists (simple list)

```
1 ---
2 actors_list:
3 - Cruise
4 - DownieJr
5 - Redford
6 - Williams
7 - Pitt
```

3) YAML dictionary (simple dictionary)

```

1  ---
2  # Actor record
3  actors:
4      name: "Tom Cruise"
5      age: 56
6      birthplace: "Syracuse, NY"
7      birthdate: "July 3, 1962"
8      photo: "https://jsonformatter.org/img/tom-cruise.jpg"

```

4) YAML dictionary (a simple list of dictionaries)

```

1  ---
2  # Actor records
3  actors_record:
4  -   name: "Tom Cruise"
5      age: 56
6      birthplace: "Syracuse, NY"
7      birthdate: "July 3, 1962"
8      photo: "https://jsonformatter.org/img/tom-cruise.jpg"
9
10 -   name: "Robert Downey Jr."
11     age: 53
12     birthplace: "New York City, NY"
13     birthdate: "April 4, 1965"
14     photo: "https://jsonformatter.org/img/Robert-Downey-Jr.jpg"
15
16

```

Let's see an heterogenous real example:

ray: "a drop of golden sun"	# key-value string
pi: 3.14159	# key-value floating point number
xmas: true	# key-value Boolean (on also valid)
bar: .inf	# key-value infinite
plop: .NaN	# key-value "not a number"
psop: null	# key-value "none".
french-hens: 3	# key-value integer
calling-birds:	# key-value array of strings
- huey	# each member is started with -
pepe	# no need to use quotes for strings
- luey	
fred	
xmas-fifth-day:	# nesting of key-values



```
calling-birds: four
partridges:
  count: 1
  location: "a pear tree"
turtle-doves: two
```

CSV

The so-called **CSV** (Comma Separated Values) format is the most common import and export format for spreadsheets. CSV format was used for many years prior to attempts to describe the format in a standardized way in **RFC 4180**.

The lack of a well-defined standard means that subtle differences often exist in the data produced and consumed by different applications. These differences can make it annoying to process CSV files from multiple sources. Still, while the delimiters and quoting characters vary, the overall format is similar enough that it is possible to write a single module which can efficiently manipulate such data, hiding the details of reading and writing the data from the programmer.

Usually the first line in a CSV file contains the table column labels. Each of the subsequent lines represent a row of the table. Commas separate each cell in the row, which is where the name comes from.

Here is an example of a CSV file. The example has four columns, labeled 'name', 'age', 'birthplace' and 'birthdate'. It has three rows including the header row.

```
Name, age, birthplace, birthdate
"Tom Cruise", 56, "Syracuse, NY", "July 3, 1962"
"Robert Downey Jr.", 53, "New York City, NY", "April 4, 1965"
```

MONGODB

MongoDB was designed from its inception to be a database focused on delivering great development experience. JSON's ubiquity made it the obvious choice for representing data structures in MongoDB.

However, there are several issues that make JSON less than ideal for usage inside of a database:

1. JSON only supports a limited number of basic data types. Most notably, JSON lacks support for dates and binary data.
2. JSON objects and properties don't have fixed length which makes traversal slower.

In order to make MongoDB JSON-first, but still with high performance and general purpose, **BSON** was invented to bridge the gap. BSON is a binary representation to store data in JSON format, optimized for speed, space, and efficiency. It's not dissimilar from other binary interchange formats like Protocol Buffers, or Thrift, in terms of approach.

MongoDB stores data in BSON format both internally, and over the network, but it does not mean you cannot think of MongoDB as a JSON database. Anything you can represent in JSON can be natively stored in MongoDB and retrieved just as easily in JSON.

A record in MongoDB is a **document**, which is a data structure composed of field and value pairs. MongoDB documents are similar to JSON objects. The values of fields may include other documents, arrays, and arrays of documents. Next example clearly reminds a JSON open data format explained before:

```
1 {  
2   "firstname": "Tom",  
3   "lastname": "Cruise",  
4   "groups": [ "News" , "Cinema" ]  
5 }
```

The advantages of using documents are:

- Documents correspond to native data types in many programming languages.
- Embedded documents and arrays reduce need for expensive joins.
- Dynamic schema supports fluent polymorphism.

MongoDB stores documents in **collections**. Collections are analogous to tables in relational databases.

The MongoDB Query API supports **create**, **read**, **update**, and **delete** operations (**CRUD**), as well as data aggregation, text search, and geospatial queries.

MongoDB has two types of interfaces. Both can create data directly or import data from either a **JSON** or **CSV** file:

- Mongo Shell, which is a Command Line Interface (CLI).
- Mongo Compass, which is a Graphic User Interface (GUI).

INSTALL MONGODB COMMUNITY EDITION

Follow these steps to install MongoDB Community Edition using the apt package manager. From a terminal, install gnupg and curl if they are not already available:

```
$ sudo apt update
```

```
$ sudo apt install -y gnupg curl
```

MongoDB's current documentation explicitly lists Ubuntu 24.04 Noble as supported for MongoDB 8.0. The official APT repository is available for Noble. So, the first step is to import the MongoDB public GPG key for version 8:

(NOTE: MongoDB's official documentation says Linux kernels **6.19 through 7.0.13 are incompatible** with MongoDB because of a TCMalloc issue, so before you do the following please check your kernel version in advance to see if the kernel is below 6.19 or beyond 7.0.13. You can check the kernel version by using command `uname -r`, if your kernel incompatible with MongoDB, please contact teaching assistant)

```
$ curl -fsSL https://pgp.mongodb.com/server-8.0.asc | sudo gpg -o  
/usr/share/keyrings/mongodb-server-8.0.gpg --dearmor
```

Next step is to create the list file for MongoDB version 8:

```
$ echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-  
8.0.gpg ] https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 multiverse" |  
sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list
```

Check all is correctly done with:

```
$ cat /etc/apt/sources.list.d/mongodb-org-8.0.list
```

Issue the following command to reload the local package database:

```
$ sudo apt update
```

You should see MongoDB being retrieved from:

```
repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0
```

If you get any error here, stop and don't install MongoDB yet. The error will usually indicate a repository or GPG-key problem.

Important: For other operative system different to Debian or a different version, check the right syntax to install the public GPG key and to create the list file.

Check which MongoDB version APT will install with:

```
$ apt-cache policy mongodb-org
```

Verify the candidate version for MongoDB is 8.0.xx. If so, proceed to install:

```
$ sudo apt install -y mongodb-org
```

The official mongodb-org package installs the MongoDB server and associated components. Check that everything is fine by displaying the version:

```
$ mongod --version
```

Enable and start MongoDB by typing:

```
$ sudo systemctl enable mongod
```

```
$ sudo systemctl start mongod
```

Now, check that MongoDB is correctly running by requiring its status:

```
$ systemctl status mongod
```

You should see: “Active: active (running)”. Otherwise, you must review all steps before or trying to guess the problem by checking the MongoDB log at `/var/log/mongodb/mongod.log`. Sometimes, the problem has to do with the Unix domain socket that MongoDB creates so that local programs on the same Linux machine can communicate with the MongoDB server.

MONGO SHELL

Mongo Shell is a command-line program (CLI) for interacting with MongoDB deployments locally or on another remote host. Use the MongoDB Shell to test queries and interact with MongoDB database using MongoDB commands. To **execute** Mongo Shell from your command prompt, just type:

```
$ mongosh
```

After that, a **CLI** will come up for you to start work with MongoDB data. The “test>” prompt means you're now inside MongoDB Shell. Maybe, you can find some startup warnings. Usually, they are not important.

Type **help** after the “test>” prompt to gain access to a short list of options. To learn more, you can find an interesting manual on how to connect to MongoDB and how to perform CRUD operations with Mongo Shell at:

<https://www.mongodb.com/docs/mongodb-shell/>

MONGO COMPASS

Mongo Compass is a free interactive **GUI** tool for querying, optimizing, and analyzing your MongoDB data in a visual and easy to use environment. Main features of Mongo Compass are:

- **Visual Representation:** Visually represents database schemas, collections and documents, making it easier to understand the data structure.
- **Query Builder:** Its query builder allows users to construct complex queries using a point-and-click interface and eliminating the need for manual MongoDB query syntax.
- **Index Management:** MongoDB Compass simplifies index management, enhancing database performance by allowing users to create, view and analyze indexes effortlessly.
- **Data Visualization:** Built-in data visualization tools enable users to analyze data using charts, graphs and histograms, facilitating insights discovery.
- **Real-Time Performance Monitoring:** MongoDB Compass allows users to monitor database performance in real-time, helping identify and resolve bottlenecks promptly.
- **Document Validation:** Compass includes a document validation feature that enables users to define and enforce data validation rules ensuring data integrity.
- **Data Exploration:** Visualize and analyze our data with tools for filtering, sorting, and aggregating.

To learn more about Mongo Compass, come to the next web site:

<https://www-mongodb.com/docs/compass/>

By default, MongoDB Compass is not included in MongoDB, so you have to download the installer of the package file:

```
$ wget https://downloads.mongodb.com/compass/mongodb-compass_1.44.3_amd64.deb
```

And install the downloaded file with:

```
$ sudo apt install ./mongodb-compass_1.44.3_amd64.deb
```

Once successfully installed, search for MongoDB Compass in the applications' menu of the computer's Desktop. Finally, remove the installer file. The use of MongoDB Compass in this Workshop is not mandatory.

WORKSHOP ACTIVITIES

There are two main variations for data exchange in 5G. Firstly, the content for an HTTP/2 PATCH request is written using standards known as JSON Patch and JSON Merge Patch. These express modifications that the client wishes to apply to a JSON document, the main distinction being that JSON Patch allows modification to the individual elements in an array, whereas JSON Merge Patch does not.

Secondly, network functions sometimes exchange content by means of binary encoded data. For example, the SMF uses binary coding to send 5G session management signalling messages to the AMF, which relays them transparently to the mobile.

In addition, the Open5GS simulator employed in this course, use YAML files to inject the configuration parameters to the 5G network functions.

In this workshop, you will be familiarized with both the use of JSON and YAML files, as well as the use of MongoDB to import and export json and csv files.

1) Create a JSON file from scratch (30%)

Using a common editor such as nano or any other plain text suitable to you, create a json file named **movistar.json**, with a single object named as “users” made of an array of 3 name/value json objects. Everyone will represent a PLMN subscriber with next fields:

- idUE: number referring the Mobile Subscription Identification Number (MSIN)
- nameUE: string referring the UE model
- endpointUE: string referring the communications channel interface
- validUE: boolean referring whether the UE is allowed or not

By now, introduce values for idUE and nameUE at your convenience, declare all endpointUE as “void”, and select two objects with validUE=true and one with validUE=false. Note that all json files use the extension json and all idUE numbers must start with a digit **different to zero**.

Using **two spaces** for JSON indentation is the widespread industry standard for web APIs, JavaScript tools, and modern editors like VS Code and Prettier. It keeps files compact while maintaining a clear, readable visual hierarchy for nested data.

You can do a fast and easy checking of your json file with some online tools like **JSONLint**. Here, you can copy and paste, directly type, or just input a URL in the editor to let JSONLint tidy and validate your JSON code: <https://jsonlint.com>.

Once you find that everything is fine, copy and paste your final json file in the box right down, at next page.

```
{
  "users": [
    {
      "idUE": 1000000001,
      "nameUE": "Samsung Galaxy S24",
      "endpointUE": "void",
      "validUE": true
    },
    {
      "idUE": 1000000002,
      "nameUE": "Apple iPhone 15",
      "endpointUE": "void",
      "validUE": true
    },
    {
      "idUE": 1000000003,
      "nameUE": "Xiaomi Redmi Note 13",
      "endpointUE": "void",
      "validUE": false
    }
  ]
}
```

2) Create a YAML file from JSON (20%)

There are several open-source tools to parse from json files to yaml files and vice versa. Here, we will be using “**yq**”, developed by Mike Farah. It is a lightweight and portable command-line YAML, JSON and XML processor written in GO language. You can find the **user manual** at:

<https://mikefarah.gitbook.io/yq>

To directly install the “yq” bin file from GitHub, type:

```
$ sudo wget -qO /usr/bin/yq \
```

```
https://github.com/mikefarah/yq/releases/latest/download/yq\_linux\_amd64
```

Once downloaded, you may need to provide execute permission to run the yq tool as a command:

```
$ sudo chmod a+x /usr/bin/yq
```

You can test the right operation of the command line tool by inquiring for the version:

```
$ yq --version
```

Now, everything is ready to parse the json file from the former section to yaml as next:

```
$ yq -Poy movistar.json
```

This will display the yaml format on the screen. To save the standard tty output to a file, use:

```
$ yq -Poy movistar.json > movistar.yaml
```

Copy and paste the obtained yaml file into the next text box:

```
users:
  - idUE: 1000000001
    nameUE: Samsung Galaxy S24
    endpointUE: void
    validUE: true
  - idUE: 1000000002
    nameUE: Apple iPhone 15
    endpointUE: void
    validUE: true
  - idUE: 1000000003
    nameUE: Xiaomi Redmi Note 13
    endpointUE: void
    validUE: false
```

Confirm whether the result matches the syntax explained in the introductory section. What kind of yaml data format is it? Furthermore, you can also use online tools to verify your yaml file. Once again, a popular one is **YAML Lint**, which is available at: <https://www.yamllint.com>.

Refer the difference in the idUE values between the json and the yaml files and propose a solution to keep the uniformity.

Syntax: yes, the result follows the YAML rules of the introduction. There are no braces, brackets or commas anymore, the structure is only given by the indentation (2 spaces), every element of the list starts with "-" and the strings are written without quotes. yamllint says the file is valid, it only gives a warning because yq does not write the "---" at the start (it is optional).

Data format: it is a dictionary (mapping) with one key "users" and its value is a list (sequence) of 3 dictionaries, one per UE. So it is the case "a simple list of dictionaries" of the introduction. Inside each dictionary the values are scalars: idUE is an integer, nameUE and endpointUE are strings and validUE is a boolean.

idUE: in the JSON file we can see the type of each value because the strings are between quotes and idUE is not. In the YAML file nothing has quotes, so idUE is just read as a number (!int) and not as an identifier. With my values the number stays the same, but the MSIN is an identifier, not a quantity, and it could start with 0. If we write 0123456789, yq gives 123456789 (the first zero is lost) and with a very long number like 12345678901234567890 yq gives 1.2345678901234567e+19. This is also why in Open5GS the mcc is written with quotes (mcc: '999').

Solution: handle idUE as a string. We can put it between quotes directly in the JSON file ("idUE": "1000000001"), or ask yq to convert it when we make the YAML file:

```
$ yq -oy '.users[].idUE |= to_string' movistar.json
users:
  - idUE: "1000000001"
    nameUE: Samsung Galaxy S24 (...)
```

In the reverse way, you can also convert a yaml file to json. Make a try with:

\$ yq -Poj movistar.yaml

Finally, find out how to convert the json file to csv. If an error comes up, propose a solution. No need to save a file. Copy the echo on the screen and paste it in the box below:

```
{
  "users": [
    {
      "idUE": 1000000001,
      "nameUE": "Samsung Galaxy S24",
      "endpointUE": "void",
      "validUE": true
    },
    {
      "idUE": 1000000002,
      "nameUE": "Apple iPhone 15",
      "endpointUE": "void",
      "validUE": true
    },
    {
      "idUE": 1000000003,
      "nameUE": "Xiaomi Redmi Note 13",
      "endpointUE": "void",
      "validUE": false
    }
  ]
}
```

First try (same idea as before, with -oc for csv output):

```
$ yq -Poc movistar.json
Error: csv encoding only works for arrays, got: !!map
```

Why: the root of movistar.json is an object (a map with only the key "users"). A CSV file is a table, so yq needs an array of objects where every object becomes one row. It does not know what to do with the map.

Solution: select the array first with the expression '.users', then each UE becomes one line and the keys are used for the header:

```
$ yq -Poc '.users' movistar.json
idUE,nameUE,endpointUE,validUE
1000000001,Samsung Galaxy S24,void,true
1000000002,Apple iPhone 15,void,true
1000000003,Xiaomi Redmi Note 13,void,false
```

The output follows RFC 4180: one header line with the field names, then one record per line with the same number of fields, separated by commas. No value has a comma, a quote or a line break inside, so yq does not need to put quotes. The only small difference is that yq ends the lines with LF (Linux) and the RFC says CRLF, but spreadsheets open it without problem.

3) Create a MongoDB document from JSON (30%)

Now, we are going to import our **movistar.json** file to MongoDB. But, previously, let's refer a particularity about every new document added to MongoDB. Every new document is labeled with an **ID**, which is composed of 12 bytes: a 4-byte timestamp that represents ID's creation measured in seconds since the Unix epoch, a 5-byte random generated value that is unique to the machine and process, and a 3-byte incrementing counter.

The easiest way to create a MongoDB document is by importing a json file. This operation can be done directly from the Linux command prompt or from the Mongo Compass GUI. You can try both.

From Mongo Compass, it is just a matter of creating a new collection into an already existing database. Once done, use the "Add Data" option to import the former json file to a new collection.

On the other hand, importing a json file from the Linux command prompt is also as easy as pie. Once again, select an already existing database in MongoDB and use the command-line tool "**mongoimport**" to import the json file to a new collection. Note that your json file should contains one object made of one array structure. Follow the instructions at: <https://www.mongodb.com/docs/database-tools/> to learn about the syntax.

Use "users" as the name of the **database** and "movistar" as the name of the **collection**.

After success, the brand-new imported collection should come up in your Mongo Shell CLI and in your MongoDB Compass GUI. Next, type the command you employed for importing and then paste the screenshot of the list of documents in your new collection created at Mongo Shell:

```
$ mongoimport --db users --collection movistar --file movistar.json
2026-09-24T18:21:46.885+0000 connected to: mongodb://localhost/
2026-09-24T18:21:46.898+0000 1 document(s) imported successfully. 0
document(s) failed to import.
```

```
$ mongosh
test> use users
switched to db users
users> show collections
movistar
users> db.movistar.find()
[
  {
    _id: ObjectId('6ab56a3a9a350c88faca86fd'),
    users: [
      {
        idUE: 10000000001,
        nameUE: 'Samsung Galaxy S24',
        endpointUE: 'void',
        validUE: true
      },
      {
        idUE: 10000000002,
        nameUE: 'Apple iPhone 15',
        endpointUE: 'void',
        validUE: true
      },
      {
        idUE: 10000000003,
        nameUE: 'Xiaomi Redmi Note 13',
        endpointUE: 'void',
        validUE: false
      }
    ]
  }
]
users> db.movistar.countDocuments()
1
```

4) Create a JSON file from MongoDB (20%)

The opposite to import data into MongoDB is to export a collection to an open standard file like csv o json. MongoDB comes with the command-line tool “**mongoexport**” to proceed with that operation. Follow the instructions at: <https://www.mongodb.com/docs/database-tools/> to learn about the syntax.

Type the command required to export all fields from your previous MongoDB collection to a new json file. Name it as **users.json**. Afterwards, paste the content of the resulting file into the text box below.

```
$ mongoexport --db users --collection movistar --out users.json
2026-09-24T18:21:54.474+0000 connected to: mongodb://localhost/
2026-09-24T18:21:54.476+0000 exported 1 record

$ cat users.json
```

```
{ "_id": { "$oid": "6ab56a3a9a350c88faca86fd" }, "users":
[ { "idUE": 1000000001, "nameUE": "Samsung Galaxy
S24", "endpointUE": "void", "validUE": true }, { "idUE": 1000000002, "nameUE": "Apple
iPhone 15", "endpointUE": "void", "validUE": true },
{ "idUE": 1000000003, "nameUE": "Xiaomi Redmi Note
13", "endpointUE": "void", "validUE": false } ] }
```

Compare the original **movistar.json** file with the new **users.json** file and observe the similitudes and differences between them. Provide your own comments below.

Similarities: the data is the same. We find the key "users" with the array of the 3 UEs in the same order, with the same 4 keys and the same values. The types are also kept: idUE is still a number (not a string), validUE is still a boolean (true/false) and nameUE and endpointUE are strings. The file is still valid JSON (I checked it with python3 -m json.tool).

Differences:

1) New field _id. MongoDB added the field "_id" to the document. It is written as {"\$oid": "..."} because an ObjectId is a BSON type that does not exist in normal JSON, so mongoexport uses the MongoDB Extended JSON format to keep the type. With the explanation of the workshop we can read the 12 bytes of 6ab56a3a9a350c88faca86fd: 6ab56a3a is the timestamp, 0x6ab56a3a = 1790274106 s since the Unix epoch = 24/09/2026 18:21:46 UTC, which is exactly the time of my mongoimport. Then 9a350c88fa is the 5-byte random value and ca86fd is the counter.

2) Presentation. users.json is written on one single line without spaces or indentation (mongoexport writes one document per line), and movistar.json had 2 spaces of indentation. It is less readable for us but smaller. With --pretty (and --jsonArray) we get an indented file again.

3) Number of documents. mongoexport says "exported 1 record": the whole movistar.json was imported as only 1 document which contains the array, not 3 documents (one per UE).

This is not very practical in a database, for example to search one UE with `find()`. A better way is to import only the array with `--jsonArray`, then we get 3 documents, each one with its own `_id`:

```
$ yq -Poj '.users' movistar.json > ue.json
$ mongoimport --db users --collection movistar --jsonArray --file ue.json
... 3 document(s) imported successfully. 0 document(s) failed to import.
```

4) Key order. the order of the keys is the same here, but a JSON object is unordered so we should not depend on it. With 3 separate documents, `mongoexport` can even give them in another order (I saw it when I exported the 3 documents to csv with `--type=csv`).

FINAL COMMENTS

Link with 5G: the idUE values are MSINs. For a Movistar subscriber in Spain the IMSI is MCC + MNC + MSIN = 214 + 07 + 1000000001, so the SUPI of my first UE would be imsi-214071000000001 (15 digits, the maximum for an IMSI). The MCC 214 for Spain is in the ITU-T E.212 list. This is another reason to keep idUE as a string, because an identifier like this must never lose a digit.

In this workshop I saw that the same information can be written in different open formats. JSON is the one used between the network functions of the 5G core (SBA), YAML is easier to read and to write by hand, this is why Open5GS and UERANSIM use it for the configuration files, and CSV is only good for flat tables. yq was very useful to pass from one format to another, but we have to be careful with the types (numbers vs strings) and with the structure (object vs array).

MongoDB stores the documents in BSON but mongoimport and mongoexport work with JSON (or CSV), so the data goes in and out without changes, only the _id is added. Before installing MongoDB 8.0 I checked the kernel with uname -r (it must not be between 6.19 and 7.0.13). I did the activities with the command line (mongosh), Compass was not mandatory.

Files of the workshop: movistar.json, movistar.yaml and users.json.